

Lab 5 C to Assembly

AIM: In this lab, the focus will be on analyzing various C code snippets and translating them into equivalent MIPS assembly language, providing a comprehensive understanding of the conversion process between high-level C programming constructs and low-level MIPS instructions.

Points to Remember:

Directives (instructions to assembler, not MIPS instructions)

```
.text      # following instructions placed in text
.data      # following objects placed in data

.globl     # make symbol available globally

a: .space 18  # uchar a[18]; or uint a[4];
   .align 2   # align next object on 22-byte addr

i: .word 2    # unsigned int i = 2;
v: .word 1,3,5 # unsigned int v[3] = {1,3,5};
h: .half 2,4,6 # unsigned short h[3] = {2,4,6};
b: .byte 1,2,3 # unsigned char b[3] = {1,2,3};
f: .float 3.14 # float f = 3.14;

s: .asciiz "abc"
   # char s[4] {'a','b','c','\0'};
t: .ascii "abc"
   # char s[3] {'a','b','c'};
```

Example:

C	Simplified C	MIPS Assembler
-----	-----	-----
int x = 5;	int x = 5;	x: .word 5
int y = 3;	int y = 3;	y: .word 3
int z;	int z;	z: .space 4
	int t;	...
		lw \$t0, x
		lw \$t1, y
z = 5*(x+y);	t = x+y;	add \$t0, \$t0, \$t1
		li \$t1, 5
	t = 5*t;	mul \$t0, \$t0, \$t1
	z = t;	sw \$t0, z

Task 1: Complete the table with Actual Instructions corresponding pseudo instructions(Use MIPS register name in your record /examples for demonstration

Pseudo Instruction	Meaning	Actual Instruction Expansion
move rd, rs	Copy register	
li rt, imm	Load immediate	
la rt, label	Load address	
nop	No operation	
clear rd	Set register to 0	
not rd, rs	Bitwise NOT	
neg rd, rs	Arithmetic negation	
b label	Unconditional branch	
beqz rs, label	Branch if zero	
bnez rs, label	Branch if not zero	
blt rs, rt, label	Branch if less than	
ble rs, rt, label	Branch if $\leq$	
bgt rs, rt, label	Branch if $>$	
bge rs, rt, label	Branch if $\geq$	
mul rd, rs, rt	Multiply (lower 32 bits)	
div rd, rs, rt	Divide quotient	
rem rd, rs, rt	Remainder	
seq rd, rs, rt	Set if equal	
sne rd, rs, rt	Set if not equal	

Task 2:

Translate the following statement into MIPS assembly code. Assume that x, y, z, q are stored in registers \$s1-\$s4. You may use the other registers to hold intermediate results.

x = x + y + z - q;

Points to Remember:

C	Simplified C	MIPS Assembler
-----	-----	-----
int x = 5;	int x = 5;	x: .word 5
int y = 3;	int y = 3;	y: .word 3
int z;	int z;	z: .space 4
	int t;	...
		lw \$t0, x
		lw \$t1, y
z = 5*(x+y);	t = x+y;	add \$t0, \$t0, \$t1
		li \$t1, 5
	t = 5*t;	mul \$t0, \$t0, \$t1
	z = t;	sw \$t0, z

Task 3

Write a MIPS assemble program that asks the user to enter two integers: n1 and n2 and prints the sum of all numbers from n1 to n2. For example, if the user enters n1=3 and n2=7, then the program should display the sum as 25. give

Task 4

Consider the following C code and partial implementation in a MIPS processor, what are the instructions that we should put in the missing section.

for(i = 0; i < 100; i++)
A[i] = i;

 add \$t0, \$zero, \$zero
 addi \$t1, \$zero, 100
 LOOP: **Missing**

addi \$t0, \$t0, 1

```
beq $t0, $t1, LOOP
```

Exercise: Identify errors in these codes and correct for codes in Lab5.zip

Task 5

Simulate the following C snippets with appropriate initialization, where **a** and **b** are integer arrays whose base addresses are in **\$a0** and **\$a1** respectively. The value of **n** is in **\$a2**. Verify for various array size(n)

```
for (i=0; i<n; i++) {
    if (i > 2) {
        a[i] = a[i-2] + a[i-1] + b[i];
    }
    else {
        a[i] = b[i]
    }
}
```

```

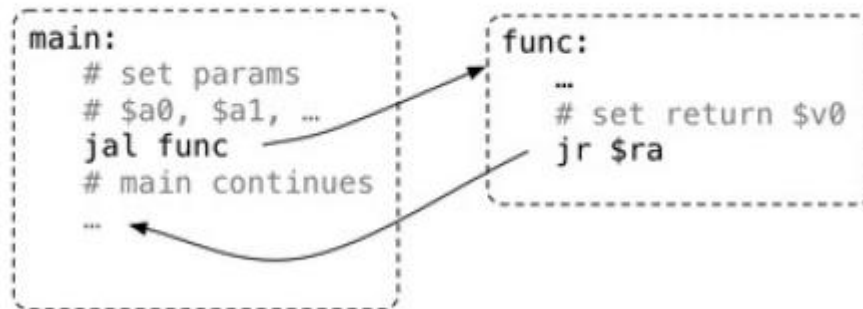
        li      $t0, 0           # $t0 = i = 0
        beq     $a2, $0, skip    # skip loop if n is zero
loop:    lw      $t1, 0($a1)      # $t1 = b[i]
        bgt     $t0, 2, else     # if (i>2) goto else
        lw      $t2, -8($a0)     # $t2 = a[i-2]
        lw      $t3, -4($a0)     # $t3 = a[i-1]
        addu    $t2, $t2, $t3    # $t2 = a[i-2]+a[i-1]
        addu    $t1, $t2, $t1    # $t1 = a[i-2]+a[i-1]+b[i]
else:    sw      $t1, 0($a0)      # a[i] = $t1
        addiu   $a0, $a0, 4      # advance array a pointer
        addiu   $a1, $a1, 4      # advance array b pointer
        addiu   $t0, $t0, 1      # i++
        bne     $t0, $a2, loop
skip:
```

Task 6

Function Implementation

Points to Remember:

```
jal label      # make a subroutine call
               # save PC in $ra, set PC to &label
               # use $a0,$a1 as params, $v0 as return
```



Consider the following C function and its MIPS implementation. Identify errors if any

```
int addfun(int a, int b)
{
    int res = a + b;
    return res;
}

int t = 0;
for (int i=0;i<10;i+=2)
{
    t += addfun(A[i],A[i+1]);
}
```

```

1
2      .text
3  main:
4      la $s7, A
5      li $s0, 0 #i
6      li $s1, 0 #res
7      li $s6, 9
8
9  loop:
10     sll $t0, $s0, 2
11     add $t0, $t0, $s7
12     lw $a0, 0($t0)
13     lw $a1, 4($t0)
14     jal addfun
15     add $s1, $s1, $v0
16     addi $s0, $s0, 2
17     bgt $s0, $s6, done
18     j loop
19  done:
20     li $v0, 10
21     syscall
22  addfun:
23     add $v0, $a0, $a1
24     jr $ra
25

```

Record your answer

Task 7

macro

Patterson and Hennessy define a **macro** as *a pattern-matching and replacement facility that provides a simple mechanism to name a frequently used sequence of instructions*. This permits the programmer to specify the instruction sequence by invoking the macro. This requires only one line of code for each use instead of repeatedly typing in the instruction sequence each time. It follows the axiom "define once, use many times," which not only reduces the chance for error but also facilitates program maintenance.

Macros are like procedures (subroutines) in this sense but operate differently than procedures. Procedures in MIPS assembly language follow particular protocols for procedure definition, call and return. Macros operate by substituting the macro body for each use at the time of assembly. This substitution is called *macro expansion*.. They do not require the protocols and execution overhead of procedures.

Study examples macros : Create and record one example of macro usage.

```

1  .include "macro_print_str.asm"
2  .data
3  .text
4  main:
5  print_string("please input an integer : ")
6  li $v0,5
7  syscall
8  move $t0,$v0
9  nor $t1,$zero,$zero
10 srl $t2,$t1,31
11 and $a0,$t2,$t1
12 print_string("it is an odd number (0: false,1:true) : ")
13 li $v0,1
14 syscall
15 end
16

```

```

| macro print_string($str)
| .data
| pstr: .asciiz $str
| .text
| la $a0,pstr
| li $v0,4
| syscall
| .end_macro

| .macro end
| li $v0,10
| syscall
| .end_macro

```

Task 8

Consider the MIDI music code written by Luke Tannenbaum (lab4_Macr0-music.asm). Attempt your own song.

Additional Optional Exercises:

.data

var1: .byte 'A', 'E', 127, -1, '\n'

var2: .half -10, 0xffff

var3: .word 0x12345678

Var4: .word 0:10

var5: .float 12.3, 0.1

var6: .double 0.1

str1: .ascii "A String\n"

str2: .asciiz "NULL Terminated String"

array: .space 100

name: .ascii "Your name"

Consider the above snippet of code and fill the following table with label/ Symbol and its Address

Memory address (starting 0x10010000)	Content

Study and Assemble the codes given (fill the table below for each of the programs)

Task x: Translation between C and MIPS

Assume: $f \rightarrow \$s0$, $g \rightarrow \$s1$, $h \rightarrow \$s2$, $i \rightarrow \$s3$ Base of array A $\rightarrow \$s6$

$f = g + A[h + 2*i];$

Sample Translation:

```
sll $t0, $s3, 1    # t0 = 2*i

add $t0, $t0, $s2  # t0 = h + 2*i

sll $t0, $t0, 2    # t0 = (h + 2*i)*4

add $t0, $t0, $s6  # t0 = &A[h + 2*i]

lw  $t1, 0($t0)    # t1 = A[h + 2*i]

add $s0, $s1, $t1  # f = g + A[h + 2*i]
```

Task x: Translation to machine Code:

```
1  .data
2  result: .word 0
3  .text
4  main:
5      li    $a0, 5        # argument n = 5
6      jal   fact          # call fact(5)
7      sw    $v0, result   # store result (120)
8      li    $v0, 10       # exit
9      syscall
10 fact:
11     addi   $sp, $sp, -8  # create stack frame
12     sw     $ra, 4($sp)   # save return address
13     sw     $a0, 0($sp)   # save n
14     li     $t0, 1
15     ble    $a0, $t0, base_case
16     addi   $a0, $a0, -1  # n = n - 1
17     jal    fact          # recursive call
18     lw     $a0, 0($sp)   # restore n
19     mul    $v0, $a0, $v0 # n * fact(n-1)
20     j      end_fact
21 base_case: li    $v0, 1    # return 1
22 end_fact:
23     lw     $ra, 4($sp)   # restore return address
24     addi   $sp, $sp, 8   # pop stack frame
25     jr     $ra          # return
```

Label	Address	Code	Basic instruction	Source instruction

Submission:
Record as instructed and demonstrate.